

MicroC/OS-II용 디바이스 드라이버 작성 ③

RTOS 상에서 구현한 비동기 시리얼포트
디바이스 드라이버

임베디드 시스템에 있어서 시리얼 통신은 아주 기본적인 통신방법이다. 전경/배경 시스템에서도 인터럽트 방식의 시리얼 디바이스 드라이버를 작성해서 사용하는 것이 일반적이지만 RTOS의 기능을 이용한 인터럽트 구동 방식의 시리얼 디바이스 드라이버를 구현하면 훨씬 더 편리하게 통신할 수 있다. 이번 호에서는 이런 편리한 RTOS용 시리얼 디바이스 드라이버를 구현하는 방법에 대해 알아본다.

글: 성원호/디오이즈 대표이사
whsung@dioiz.com

두 장치(일반적으로 컴퓨터 또는 임베디드 시스템)가 멀리 떨어져 있을 때, 대부분의 경우 데이터를 직렬로 송신한다. 장치 안에서는 8비트 또는 그 이상의 비트 단위로 데이터를 처리하기 때문에 송신하는 병렬에서 직렬로, 수신하는 직렬에서 병렬로 변환할 필요가 있다. 통신 모드는 그림 1에 보인 것처럼 기본적으로 3가지가 있다.

- 심플렉스(Simplex): 한 방향(A에서 B)으로만 데이터가 전송된다. 심플렉스 링크를 사용하는 장치의 구체적인 예로 하키, 농구 등 스포츠 경기장에서 사용하는 점수판을 들 수 있다. 점수나 시간을 모니터링하는 기록원이 콘솔을 통해 입력한 정보는 관중이 볼 수 있는 디스플레이 장치로 직렬 전송된다.
- 반이중(Half-duplex): 데이터를 한 방향(A에서 B) 또는 반대방향(B에서 A)으로 전송할 수 있지만 동시에 양방향으로 전송할 수는 없다. 공장자동화에서 많이 사용하는 RS-485 인터페이스는 반이중 방식이다.
- 전이중(Full-duplex): 데이터를 동시에 양방향으로 전송할 수 있다.

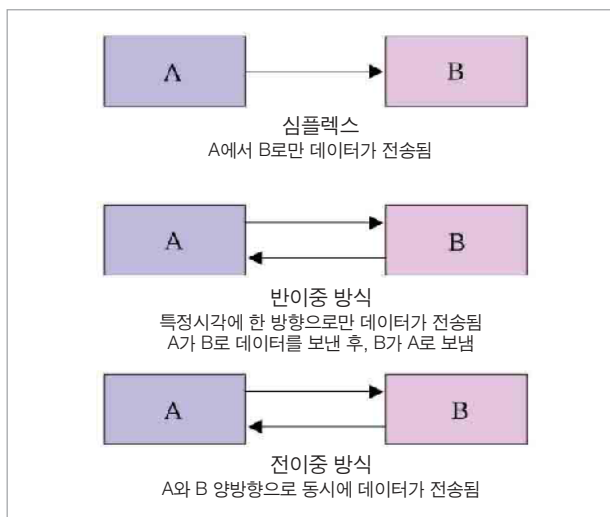


그림 1. 통신모드

여기서는 비동기 시리얼 통신 포트상에서 전이중 방식으로 데이터를 주고받는 방법에 대해 설명하지만 실제 어떤 데이터를 주고받는 것까지 설명하지는 않는다. 즉, 통신 프로토콜을 자체를 다루지는 않는다. 시리얼포트의 원리와 기본적인 설명은 참고서적[2] 또는 다른 서적을 참조하기 바란다. 'Joe Campbell

의 C Programmer's Guide to Serial Communications' [3]라는 훌륭한 책에서 비동기 시리얼 통신에 대한 거의 모든 정보를 찾을 수 있을 것이다.

비동기 통신

비동기 통신 시스템에서 두 장치 사이에 데이터가 전송될 때, 수신부의 클록은 송신부의 클록에 동기하지 않는다. 각 바이트의 전송은 시작 신호(start signal)를 선두로 해서 하나 또는 그 이상의 정지 신호(stop signal)로 종료한다. 시작과 정지 신호는 수신부에서 동기용으로 사용한다. 데이터 전송라인은 그림 2와 같이 아무것도 전송하지 않을 때 HIGH 상태이다. 송신장치는 각 바이트를 전송할 때 전송라인의 상태를 HIGH에서 LOW로 바뀌서 최초로 시작비트를 전송한다.

이 상태변화를 이용해서 1바이트 데이터 전송이 시작된다는 것을 수신장치로 알려주는 것이다. 수신장치는 시작비트와 데이터 비트를 감지한다. 바이트 전송이 끝날 때, 하나 또는 그 이상의 정지비트에 의해 전송라인은 다시 HIGH 상태가 된다. 이 시점에서 송신부는 다음 바이트를 전송할 수 있다. 시작비트와 정지비트는 수신기가 바이트 단위로 송신기에 동기할 수 있도록 해준다. 그림 2에서 바이트 전송은 최하위 비트부터 시작되는 것에 주목하라. 또 각 바이트를 전송할 때 동기화 목적으로 2비트 이상이 필요하다. 따라서 동기용 비트는 시스템에 20% 이상의 오버헤드를 초래한다.

송신기와 수신기는 어떤 속도로 각 비트를 전송할 지 서로 약속을 정해야 한다. 이 전송 속도를 보레이트(baud rate)라 한다. 송신기와 수신기가 같은 보레이트를 사용한다면 실제 보레이트는 어떤 값이라도 상관없다. 그러나 산업계는 표 1과 같이 보레이트를 표준화했다. 따라서 시리얼 통신에서는 표 1에 있는 전송 속도를 사용하는 것이 일반적이다.

비동기 통신은 UART(Universal Asynchronous Receiver Transmitter)라는 장치를 써서 수행한다. 데이터를 송수신할 때 프로그램은 단순히 UART로부터 한 바이트의 데이터를 읽거나 쓰면 된다. UART는 일반적으로 동시에 데이터를 보내고 받을 수 있다(전이중 방식을 지원한다). 마이크로프로세서의 입장에서 볼 때 UART는 단순히 하나 또는 그 이상의 I/O 포트나 메모리 위치로 여겨진다. UART는 송수신 상태나 진행을 확인할 수 있는 하나 또는 그 이상의 상태 레지스터(status register)를

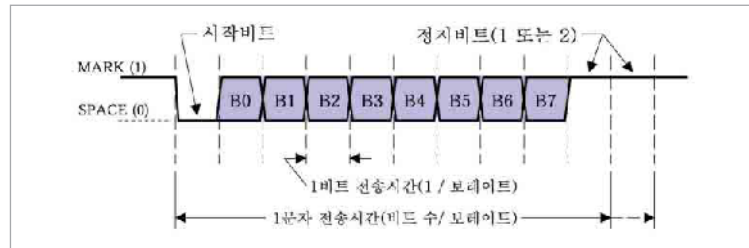


그림 2. 비동기 통신 타이밍 다이어그램

표 1. 표준 보레이트

보레이트	비트 전송시간(μs)	1초당 바이트 수 (노트 1)	바이트 전송 시간(μs) (노트 1)
300	3,333.3	30	33,333
600	1,666.6	60	16,667
1200	833.3	120	8,333
2400	416.7	240	4,167
4800	208.3	480	2,083
9600	104.2	960	1,042
19200	52.1	1920	521
38400	26.0	3840	260
56000	17.9	5600	179

가지고 있다. 따라서 마이크로프로세서는 상태 레지스터를 확인해서 데이터가 도착했는지, 통신 에러가 발생했는지, 송신이 완료됐는지 알 수 있다. 또 컨트롤 레지스터(control register)를 이용해서 UART의 동작 모드를 설정할 수 있다. UART의 설정은 보레이트 세팅, 정지 비트 세팅(1, 1과 1/2 또는 2), 데이터의 송수신이 완료됐을 때 인터럽트를 발생하도록 설정하는 것 등을 포함한다.

아마도 가장 일반적인 UART는 National Semiconductor사의 NS16550일 것이다. 현재 시장에 많은 UART가 나와 있지만 그 중 대중적인 것들은 AMD사의 Z8530, Motorola사의 6850 ACIA, Zilog사의 Z-80 SIO 등이다. NS16550은 데이터를 송수신하는 데 필요한 모든 기능을 포함하는 것은 물론, 대부분의 마이크로프로세서와의 인터페이스를 용이하게 해 주는 내장 보레이트 생성기를 가지고 있다. UART의 또 다른 장점은 많은 싱글 칩 CPU들에 내장돼 있다는 점이다. 따라서 임베디드 시스템은 터미널, 컴퓨터, 심지어 다른 임베디드 시스템과의 통신을 손쉽게 구현할 수 있다.

UART를 이용해서 송수신하는 데이터는 8비트(또는 그 이하) 또는 복수의 8비트 데이터 등으로 구성할 수 있다. 따라서 여러

LSD		MSD							
		0	1	2	3	4	5	6	7
0	0000	NUL	DLE	SP	0	8	P	-	p
1	0001	SOH	DC1	1	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	\$	4	D	T	d	t
5	0101	ENO	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	
D	1101	CR	GS	=	=	M	]	m	}
E	1110	SO	RS	>	>	N	^	n	~
F	1111	SI	US	/	?	O	_	o	DEL

그림 3. ASCII 문자 체계(7비트 코드)

분은 이진 데이터, ASCII 문자, EBCDIC(Extended Binary Coded Decimal Interchange Code), BCD(Binary Coded Decimal) 수치 등을 전송할 수 있다. 지금까지 영어권에서 사용되는 가장 중요한 문자 체계는 ASCII다. ASCII는 7비트 코드다. 7비트 이진수와 ASCII 코드의 관계를 보여주는 ASCII 표는 그림 3을 참고하기 바란다.

ASCII 문자는 C에서 문자열을 표현하는데 사용한다. 예를 들어 “HELLO” 라는 문자열은 다음과 같은 ASCII 코드로 표현한다.

ASCII:	H	E	L	L	O	\0
Binary	0x48	0x45	0x4C	0x4C	0x4F	0x00

ASCII 표에는 특수 문자에 해당하는 2개의 컬럼이 있다. 이 특수 ASCII 코드 중 몇 개는 C 프로그래머에게 잘 알려져 있다. 이들은 NUL(널 문자, 0x00), BEL(벨, 0x07), BS(백스페이스, 0x08), LF(라인 피드, 0x0A), CR(캐리지 리턴, 0x0C), FF(폼 피드, 0x0F), ESC(Esc, 0x1B), 그리고 SP(공백, 0x20) 등이다.

데이터 송수신

앞서 설명한 바와 같이 메모리 또는 I/O 포트의 특정한 위치를 읽고 씌으로써 UART를 이용한 데이터 송신과 수신에 가능하다. UART 상태 레지스터의 특정 비트를 조사하면 데이터가 수신됐는지 확인할 수 있다. 이렇게 UART의 상태를 모니터링 하는 것을 I/O 장치 폴링(polling)이라 하며 마이크로프로세서가 데이터 송수신 속도보다 빠르게 상태를 모니터링 할 수 있을 때 사용한다. 그러나 이런 폴링 방식은 마이크로프로세서가 다른 일을 수행하는 도중 수신되는 데이터를 놓칠 수 있다는 심각한 문제점이 있다. 마이크로프로세서는 시리얼 I/O 포트를 모니터링 하는 것 외에 다양한 일을 수행해야 하므로, 데이터 송수신을 처리하기 위해 인터럽트 방식을 사용하는 것이 일반적이다.

데이터 수신

인터럽트 방식을 사용할 경우, 시리얼 포트가 바이트를 수신하면 인터럽트가 발생한다. 인터럽트를 처리하는 서비스루틴은 포트로부터 바이트를 읽는다. 이때 보통 인터럽트 소스를 리셋한다. 이 시점에서 수신된 바이트를 ISR 내에서 해석, 처리할 것인지 아니면 배경 프로세스에서 처리할 수 있도록 버퍼에 저장할 것인지 결정해야 한다. 버퍼를 사용할 경우, 버퍼의 크기는 배경 프로세스가 얼마나 자주 수신 데이터를 처리할 수 있는지에 따라 달라진다. 예를 들어, 배경 프로세스가 최악의 경우 200ms 동안 데이터를 처리하지 못할 경우, 9600보레이트에서 192(960bytes/sec×200ms)바이트 크기의 버퍼를 확보해야 한다. 시리얼 포트로부터의 데이터를 저장하기 위해 링 버퍼(Ring Buffer) 또는 원형 버퍼(Circular Buffer)라 불리는 특별한 버퍼를 사용한다.

시스템에 메모리 제약이 심해서 큰 버퍼를 사용할 수 없을 경우는 흐름제어(flow control)를 사용할 수 있다. 데이터 수신 인터럽트는 버퍼가 가득 찬 것을 송신자에게 알려줄 수 있다. 송신자는 수신자가 버퍼를 비우고 다음 데이터를 받을 준비가 된 것을 알려줄 때까지 송신을 중단한다. 가장 일반적인 흐름제어 방법은 XON-XOFF라 하며 특수 ASCII 문자를 사용한다. 이 문자는 “송신하십시오”에 해당하는 XON에 DC1(0x11), “데이터를 보내지 마시오”에 해당하는 XOFF에 DC3(0x13)을 사용한다. XON-XOFF 흐름제어를 사용할 경우 바이너리 데이터를

송수신 할 수 없다. 송수신 데이터에 이 두 특수문자가 포함될 수도 있기 때문이다.

RS-232C 라인의 몇 라인을 이용해서 흐름제어를 하는 경우도 있다. 이 경우는 이진 데이터를 송신하는 것이 가능하다. 시리얼 포트를 모뎀과 연결하지 않을 경우, RS-232C 표준은 어떤 라인을 사용해서 흐름제어를 구현할 것인지 정의하지 않는다. 모뎀 제어 라인인 RTS, CTS, DSR, 그리고 DTR 중 어떤 것을 사용해도 상관없다. 따라서 여러분의 장치에서 어떤 방식으로 흐름제어를 구현할 것인지는 여러분 결정에 달려있다.

링 버퍼를 이용한 입력 버퍼링을 그림 4에 보여준다. 바이트가 수신될 때, ISR은 시리얼 포트로부터 바이트를 읽고(①), 링 버퍼에 저장한다(②). 응용프로그램(배경)은 링 버퍼를 확인해서 바이트가 수신됐는지 확인한다(③). 링 버퍼에 데이터가 있다면 가장 먼저 저장된 바이트를 링 버퍼에서 읽어온다.

ISR과 응용프로그램에서 사용하는 인터페이스 함수의 의사 코드를 리스트 1에서 보여준다.

응용프로그램이 링 버퍼에 접근하는 동안 ISR 또는 다른 인터페이스 함수가 링 버퍼에 접근하는 것을 막기 위해 인터럽트를 비활성화해야 한다는 점에 주목하라. 응용프로그램이 적절한 시기에 링 버퍼에서 데이터를 읽어가지 않을 경우, 링 버퍼는 가득 차서 수신 데이터를 잃게 된다.

수신 데이터에 대한 응답은 배경 프로세스가 얼마나 자주 실행되는가에 달려있다. 이때 실시간 커널을 사용한다면, ISR 내에서 처리하지 않고도 데이터가 수신되자마자 그에 대한 처리가 가능하다. 그렇게 하려면 그림 5와 같이 링 버퍼 관리에 세마포어를 추가해야 한다.

이 경우 응용프로그램은 먼저 세마포어에서 대기한다(①). 바이트가 수신되면 ISR은 시리얼 포트로부터 바이트를 읽고(②) 이 데이터를 링 버퍼에 저장한다(③). 그 다음 ISR은 새로운 데이터가 수신된 것을 알려주기 위해 세마포어로 신호를 보낸다(④). 세마포어로 신호를 보내면 이를 기다리던 태스크는 실행 준비 상태가 된다. ISR이 완료되면 커널은 세마포어에서 대기 중인 태스크가 CPU를 사용할 수 있는 준비가 된 최상위 우선순위 태스크인지를 결정한다. 대기 중인 태스크의 우선순위가 가장 높을 경우 ISR은 이 태스크를 실행토록 해준다(선점형 커널인 경우). 응용프로그램 즉, 시리얼 포트에 대기 중이던 태스크는 링 버퍼로부터 바이트를 읽어낸 뒤, 필요에 따라 다음 일을 수행한다.

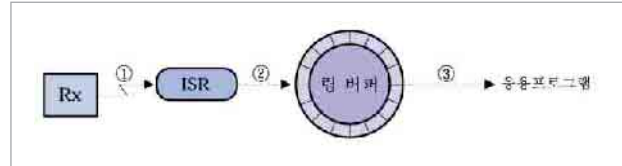


그림 4. 버퍼링 방식 시리얼 I/O, 데이터 수신

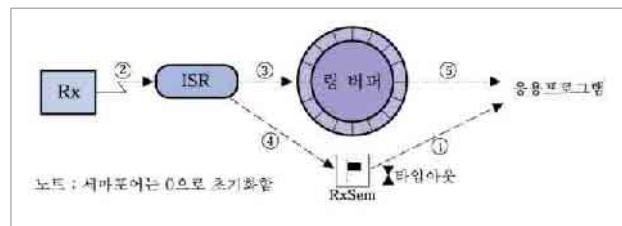


그림 5. 세마포어를 사용한 버퍼링 방식 시리얼 I/O, 데이터 수신

```
ISR CommRxISR(void)
{
    INT8U c;

    프로세서의 레지스터를 스택에 저장한다;
    c = RX 포트로부터 바이트를 읽어온다;
    if(수신 버퍼가 비어있음) {
        수신된 바이트를 링버퍼에 저장한다;
    }
    프로세서 레지스터를 스택으로부터 복구한다;
    인터럽트로부터 리턴한다;
}

INT8U CommGetChar(void)
{
    INT8U c;

    c = NUL;
    인터럽트 비활성화;
    /* 링 버퍼에 접근하는 동안 인터럽트를 금지한다 */
    if(Rx 링 버퍼 데이터가 있는가) {
        c = 링 버퍼에서 바이트를 읽어온다;
    }
    인터럽트 활성화;
    return(c);
}
```

리스트 1

ISR과 응용프로그램에서 사용하는 인터페이스 함수의 의사 코드를 리스트 2에 보여준다. 실제 ISR 코드와 인터페이스 함수는 나중에 설명한다. 이전의 방법과 마찬가지로 응용프로그램이 제때 링 버퍼에서 데이터를 읽어가지 않으면 링 버퍼는 가득 차서 수신 데이터를 잃게 된다. 그러나 실시간 커널을 사용하면 그런 상황이 발생할 확률이 줄어든다.

대부분의 실시간 커널은 수신 바이트를 기다리는 시간을 명시할 수 있다. 이를 이용하면 통신 라인이 어떤 문제가 발생한 경우 적절한 조치를 취할 수 있도록 할 수 있다. 예를 들어 태스크가 메시지를 전송하고 그 응답을 기다리는 상황을 생각해 보자. 이때 일정 시간 내에 응답이 수신되지 않는 경우, 송신자는 메시지를 기다리는 수신자가 없거나 통신라인에 이상이 발생했다는 결론을 내릴 수 있다.

```
ISR CommRxISR(void)
{
    INT8U c;

    프로세서의 레지스터를 스택에 저장한다;
    OS에게 인터럽트를 처리하고 있다는 것을 알려준다;
    c = RX 포트로부터 바이트를 읽어온다;
    if(수신 버퍼가 비어있음) {
        수신된 바이트를 링버퍼에 저장한다;
        Rx 세마포어로 신호를 보낸다;
    }
    OS에게 인터럽트 처리가 끝난 것을 알려준다;
    프로세서 레지스터를 스택으로부터 복구한다;
    인터럽트로부터 리턴한다;
}

INT8U CommGetChar(INT8U *err)
{
    INT8U c;

    세마포어에 신호가 오기를 기다린다(세마포어 타임아웃 사용);
    if(타임아웃) {
        *err = 타임아웃 에러;
```

```
return(0);
}
인터럽트 비활성화;
c = 링 버퍼에서 바이트를 읽어온다;
인터럽트 활성화;
*err = 에러 없음;
return(c);
}
```

리스트 2

모든 바이트가 수신될 때마다 세마포어로 신호를 보내는 것은 귀중한 CPU 타임을 낭비하는 일이다. 이런 낭비를 막을 수 있는 한 가지 방법은 특수한 데이터가 수신될 때만 세마포어로 신호를 보내는 것이다. 예를 들면 개행문자(CR)가 수신될 때만 세마포어로 신호를 보낼 수 있다. 응용프로그램은 완전한 명령 메시지가 도착할 때만 이에 대한 처리를 함으로써 오버헤드를 줄일 수 있다. 물론 통신 버퍼는 하나 또는 그 이상의 명령 메시지를 저장할 수 있는 충분한 크기를 확보하고 있어야 한다. 이 방법을 다음의 리스트 3에서 보여주고 있다.

데이터 송신

데이터 송신은 데이터 수신과 비슷하게 동작한다. 배경 프로세스는 데이터를 출력 버퍼로 넘겨준다. 이때 UART의 송신부가 바이트를 전송할 수 있는 상태면 송신 인터럽트를 발생시킨다. 송신 ISR은 버퍼로부터 데이터를 한 바이트 꺼내서 UART에 쓴다. 그런데 여기서 작은 문제가 발견된다. 시리얼 포트는 바이트 전송을 완료한 후에만 인터럽트를 발생시킨다는 점이다. 이 문제를 해결할 수 있는 가장 적절한 방법은 데이터를 전송할 필요가 있을 때까지 송신 인터럽트를 비활성화하는 것이다. 하나 이상의 데이터가 출력 버퍼에 저장되었을 때, 송신 인터럽트를 활성화한다. 송신 인터럽트를 활성화하자마자, ISR이 전송하고자 하는 첫 번째 바이트를 출력 버퍼에서 읽어서 UART에 쓴다. ISR은 버퍼를 조사한 후 전송할 데이터가 없으면 송신 인터럽트를 비활성화한다.

데이터 버퍼링은 파일의 내용을 전송하는 것처럼 많은 양의 데이터를 전송할 경우 큰 의미가 있다. 링 버퍼를 이용한 출력


```

ISR CommRxISR(void)
{
    INT8U c;

    프로세서의 레지스터를 스택에 저장한다;
    OS에게 인터럽트를 처리하고 있다는 것을 알려준다;
    c = RX 포트로부터 바이트를 읽어온다;
    if(수신 버퍼가 비어있음) {
        수신된 바이트를 링 버퍼에 저장한다;
        if(수신된 바이트가 명령 메시지의 종료를 알려주는 문자) {
            Rx 세마포어로 신호를 보낸다;
        }
    }
    OS에게 인터럽트 처리가 끝난 것을 알려준다;
    프로세서 레지스터를 스택으로부터 복구한다;
    인터럽트로부터 리턴한다;
}

INT8U CommGetCommand(INT8U *command, INT8U *nbytes)
{
    INT8U c;
    INT8U nrx;

    바이트가 수신되기를 기다린다(타임아웃 세마포어 사용);
    if(타임아웃) {
        *nbytes = 0;
        return(타임아웃 에러);
    }
    nrx = 0; /* 수신 데이터 카운터를 리셋한다 */
    인터럽트 비활성화;
    c = 링 버퍼에서 바이트를 읽어온다;
    while(c != 명령 종료 바이트) {
        *command++ = c; /* 명령 메시지 저장 */
        nrx++; /* 수신 데이터 카운터 증가 */
        c = 링 버퍼에서 바이트를 읽어온다;
    }
    인터럽트 활성화;
}

```

```

        *nbytes = nrx error; /* 수신된 바이트 수를 저장한다
        */
        return(No error);
    }
}

```

리스트 3

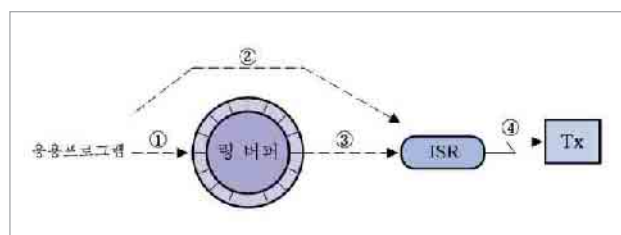


그림 6. 버퍼링 방식 시리얼 I/O, 데이터 전송

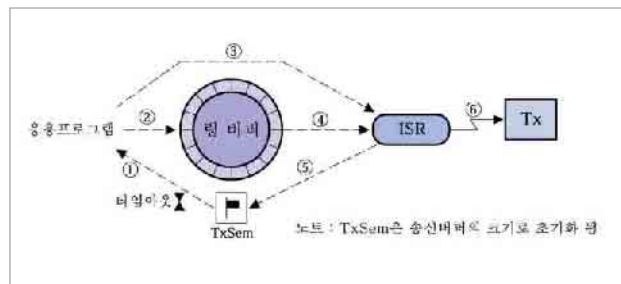


그림 7. 세마포어를 이용한 버퍼링 방식 시리얼 I/O, 데이터 전송

버퍼링을 그림 6에 표시했다. 전송하고자 하는 데이터를 링 버퍼에 저장한다(①). 한 바이트 데이터를 버퍼에 저장한 후 송신 인터럽트를 활성화한다(②). UART가 데이터를 전송할 수 있는 상태라면 인터럽트가 발생하고 ISR은 버퍼에서 가장 먼저 저장된 바이트를 읽어온다(③). 그 뒤 바이트를 시리얼 포트에 출력한다. 마지막으로 읽어온 데이터로 인해 출력 버퍼가 비면 송신 인터럽트를 비활성화한다.

ISR과 응용프로그램에서 사용하는 인터페이스 함수의 의사 코드를 리스트 4에 게재했다.

그림 7은 실시간 커널의 기능을 사용하는 방법이다. 세마포어는 링 버퍼가 가득 찼을 때 전송 태스크를 정지하는 교통 신호기로 사용된다. 태스크는 데이터를 전송하기 위해 세마포어를 기다린다(①). 링 버퍼가 가득 차지 않은 경우, 태스크는 계속 실행되어 바이트를 링 버퍼에 넘겨준다(②). 넘겨진 바이트

```

void CommPutChar(INT8U c)
{
    인터럽트 비활성화; /* 액세스 동안 인터럽트를 막는다 */
    if(Tx 링 버퍼가 비어있음) {
        링 버퍼에 전송할 데이터를 저장한다;
        if(링 버퍼에 저장되는 첫 번째 바이트) {
            Tx 인터럽트 활성화;
        }
    }
    인터럽트 활성화; /* 인터럽트를 허용한다 */
}

ISR CommTxCharISR(void)
{
    INT8U c;

    프로세서의 레지스터를 스택에 저장한다;
    if(Tx 링 버퍼에 데이터가 있는가) {
        c = 링 버퍼에서 전송할 데이터를 읽어 온다;
        전송 포트에 'c'를 쓴다;
    } else {
        Tx 인터럽트 비활성화;
    }
    프로세서 레지스터를 스택으로부터 복구한다;
    인터럽트로부터 리턴한다;
}

```

리스트 4

가 링 버퍼의 첫 번째 바이트인 경우, 송신 인터럽트를 활성화한다(③). 송신 ISR은 링 버퍼에서 가장 오래된 바이트를 빼내고(④) 링 버퍼에 다른 바이트를 저장할 수 있다는 것을 알리기 위해 세마포어로 신호를 보낸다(⑤). ISR은 바이트를 UART로 출력한다.

TxSem이 카운팅 세마포인 점은 매우 중요하며, 이 세마포어는 링 버퍼의 크기로 초기화 해야 한다. ISR과 응용프로그램에서 사용하는 인터페이스 함수의 의사코드를 다음에 보여준다. 실제 ISR 코드와 인터페이스 함수는 뒤에 설명한다.

```

void CommPutChar(INT8U c, INT8U *err)
{
    Tx 링 버퍼가 하나 이상 비기를 기다린다(타임아웃 세마포어 사용);
    if(타임아웃) {
        *err = 타임아웃 에러;
        return;
    }
    인터럽트 비활성화;
    전송하려는 바이트(c)를 Tx 링 버퍼에 저장한다;
    if(Tx 링 버퍼의 첫 번째 바이트) {
        TX 인터럽트 활성화;
    }
    인터럽트 활성화;
    *err = 에러 없음;
}

ISR CommTxCharISR(void)
{
    INT8U c;

    프로세서의 레지스터를 스택에 저장한다;
    if(Tx 링 버퍼에 데이터가 있는가) {
        c = Tx 링 버퍼에서 전송하고자 하는 다음 바이트를 가져온다;
        Tx 포트로 'c' 문자를 출력한다;
        Tx 세마포어로 신호를 보낸다;
    } else {
        TX 인터럽트를 비활성화 한다;
    }
    프로세서 레지스터를 스택으로부터 복구한다;
    인터럽트로부터 리턴한다;
}

```

리스트 5

드라이버 포팅

지난 호와 마찬가지로 드라이버의 풀 소스코드 게재는 지면 관계상 생략한다. 여기서 소개하는 드라이버의 풀 소스코드는

```

typedef struct {
    INT16U RingBufRxCtr; /* Rx 링 버퍼에 저장된 데이터 수
    */
    OS_EVENT *RingBufRxSem; /* 수신 세마포어의 포인터 */
    INT8U *RingBufRxInPtr; /* 다음 데이터를 저장할 곳을 가
리키는 포인터 */
    INT8U *RingBufRxOutPtr; /* 다음에 읽어갈 데이터를 가리
키는 포인터 */
    INT8U RingBufRx[COMM_RX_BUF_SIZE]; /* 수신용 링 버퍼
저장영역(Rx) */

    INT16U RingBufTxCtr; /* Tx 링 버퍼에 저장된 데이터 수
    */
    OS_EVENT *RingBufTxSem; /* 송신 세마포어의 포인터 */
    INT8U *RingBufTxInPtr; /* 다음 데이터를 저장할 곳을 가
리키는 포인터 */
    INT8U *RingBufTxOutPtr; /* 다음에 읽어갈 데이터를 가리
키는 포인터 */
    INT8U RingBufTx[COMM_TX_BUF_SIZE]; /* 송신용 링 버퍼
저장영역(Tx) */

} COMM_RING_BUF;

...
COMM_RING_BUF ComBuf;
COMM_RING_BUF *pbuf;
...

void CommInit(void)
{
    INT8U val;

    pbuf = &ComBuf; /* COMM1에 대한 링 버퍼 초기화 */
    pbuf->RingBufRxCtr = 0;
    pbuf->RingBufRxInPtr = &pbuf->RingBufRx[0];
    pbuf->RingBufRxOutPtr = &pbuf->RingBufRx[0];
    pbuf->RingBufRxSem = OS_SemCreate(0);
    pbuf->RingBufTxCtr = 0;
    pbuf->RingBufTxInPtr = &pbuf->RingBufTx[0];
    pbuf->RingBufTxOutPtr = &pbuf->RingBufTx[0];

```

```

        pbuf->RingBufTxSem =
OS_SemCreate(COMM_TX_BUF_SIZE);

        /* Prevent other bits of ioc1 from being altered
        */
        wsr = 15;
        val = ioc1;
        wsr = 0;
        val |= 0x20;

        sp_con = 0x09; /* mode1 & recieve enable */
        ioc1 = val; /* select TxD */

        /* Baud rate must be written in the order of LSB
        first at the same address */
        baud_rate = ((unsigned char) BAUD_REG);
        baud_rate = ((unsigned char) (BAUD_REG >> 8));
    }

```

리스트 6

실습기트(DTB-196-BK, <http://www.acornpub.co.kr> 참조)에 포함돼 있다. 또는 서적[2]를 참조해도 된다. 다음은 시리얼 드라이버용으로 사용하는 C 자료구조 형과 드라이버 초기화 함수를 보여준다.

리스트 6과 같이 시리얼 포트용 자료구조(COMM_RING_BUF)를 정의하면 특성이 같은 여러 개의 시리얼 포트를 한 드라이버 안에서 처리하는 것이 쉬워진다. 이런 개념은 다른 디바이스 드라이버를 작성할 때도 마찬가지로 적용할 수 있다.

리스트 6에 있는 코드는 80c196kc에 내장된 UART용으로 포팅한 것이다. 80c196kc에 내장된 UART는 하나 밖에 없기 때문에 여러 개의 포트를 처리할 일이 없어서 코드가 간단하다. 반면에 참고서적[2]에 있는 원래 드라이버는 PC용으로 만들었기 때문에 특성이 같은 UART를 여러 개 처리할 수 있도록 돼 있다. PC용 드라이버에 있는 각 함수들은 어떤 UART를 처리해야 할 지 전달인자를 통해 알 수 있다. 이 시리얼 디바이스 드라이버를 사용하려면 시스템 초기화시 CommInit() 함수를 호출해야 한다. 이 함수는 UART용 자료구조를 초기화하고 지정된 보레이트로 하드웨어를 설정하는 일을 한다.


```

void RxIntHandler(void)
{
    INT8U c;

    c = sbuf; /* 수신된 데이터를 읽는다 */

    if(pbuf->RingBufRxCtr < COMM_RX_BUF_SIZE) { /* 버퍼
가 가득 찼는지 점검 */
        pbuf->RingBufRxCtr++; /* 공간이 있음. 카운터를 증
가한다 */
        *pbuf->RingBufRxInPtr++ = c; /* 데이터를 버퍼에
저장 */

        if(pbuf->RingBufRxInPtr == &pbuf-
>RingBufRx[COMM_RX_BUF_SIZE]) { /* 포인터 경계 처리 */
            pbuf->RingBufRxInPtr = &pbuf->RingBufRx[0];
        }
        OSSemPost(pbuf->RingBufRxSem); /* 세마포어로 신호
를 보내 데이터가 수신된 사실을 알린다 */
    }
}

INT8U CommGetChar(INT16U to, INT8U *err)
{
    INT8U c;
    INT8U oserr;

    OSSemPend(pbuf->RingBufRxSem, to, &oserr); /* 데이터
가 수신되기를 기다린다 */
    if(oserr == OS_TIMEOUT) { /* 타임아웃이 만료되었는가?
*/
        *err = COMM_RX_TIMEOUT; /* 예, 에러코드를 리턴한다
*/
        return(NUL);
    } else {
        OS_ENTER_CRITICAL();
        pbuf->RingBufRxCtr--; /* 아니오, 데이터 카운터를
감소한다 */
    }
}

```

```

        c = *pbuf->RingBufRxOutPtr++; /* 버퍼에서 데이터
를 읽어낸다 */

        if(pbuf->RingBufRxOutPtr == &pbuf-
>RingBufRx[COMM_RX_BUF_SIZE]) { /* 포인터 경계 처리 */
            pbuf->RingBufRxOutPtr = &pbuf-
>RingBufRx[0];
        }
        OS_EXIT_CRITICAL();
        *err = COMM_NO_ERR;
        return(c);
    }
}

```

리스트 7

처음에는 송수신 버퍼가 모두 비어 있는데도 불구하고 수신용 세마포어(RingBufRxSem)의 카운트 값을 0으로, 송신용 세마포어(RingBufTxSem)의 카운트 값을 COMM_TX_BUF_SIZE로 초기화하는 것을 주목하기 바란다.

COMM_TX_BUF_SIZE는 송신용 버퍼의 바이트 단위 크기다. 이 드라이버를 호출하는 응용프로그램 태스크 입장에서 보면 수신 버퍼로부터 데이터를 읽어가는 동작과 송신버퍼로 데이터를 저장하는 동작이 가장 중요하다. 태스크가 수신 버퍼에서 데이터를 최초로 읽어가려고 시도할 때는 버퍼가 비어 있다.

따라서 태스크의 실행을 멈추고 대기상태로 만들 필요가 있다. 이 메커니즘은 수신 세마포어의 값을 0으로 초기화하고, 버퍼에서 데이터를 읽기 전에 세마포어 대기 함수를 호출함으로써 구현할 수 있다. 이를 구현한 실제 코드는 리스트 7에 게재돼 있다.

MicroC/OS-II는 인터럽트 금지/해제 방법으로 OS_ENTER_CRITICAL()과 OS_EXIT_CRITICAL() 매크로를 사용한다는 점을 주목하라. 수신 인터럽트 서비스 루틴인 RxInt Handler()는 커널한테 데이터가 수신된 사실을 알리기 위해 OSSemPost()를 호출한다. 여러 분이 다른 RTOS를 사용하고 있다면 이 함수를 적절한 함수로 바꿔줘야 한다. 시리얼 포트로부터 데이터를 읽고자 하는 태스크는 CommGetChar() 함수를 호출해야 한다. 이 함수를 호출한 태스크는 버퍼에 데이터가 저장돼 있을

경우 즉시 데이터를 한 바이트 읽어내어 호출자로 리턴한다.

하지만 데이터가 없을 경우는 OSSemPend() 함수를 호출할 때 대기상태가 된다. 이 함수를 호출한 태스크가 깨어나서 CPU 사용권을 받을 준비상태가 되는 경우는 to로 지정한 시간이 지나거나 앞에서 설명한 대로 수신 인터럽트가 발생해서 세마포어로 신호를 보내는 함수를 호출한 경우뿐이다. 어쨌든 데이터를 얻어오고자 하는 태스크가 버퍼에 데이터가 없을 때 대기상태로 들어가고, 이때 다른 태스크가 CPU를 받아서 실행할 수 있는 기회를 얻을 수 있다는 것은 RTOS를 사용할 때 누릴 수 있는 가장 큰 장점 중 하나다. 이런 메커니즘을 통해 CPU 사용률을 극대화할 수 있다.

이번에는 데이터를 송신하기 위해 송신 버퍼에 데이터를 저장하고자 하는 태스크를 고려해 보자. 물론 초기에는 송신 버퍼도 비어 있기 때문에 최초로 송신버퍼에 데이터를 저장하려고 시도하는 태스크는 버퍼 크기만큼 데이터를 써 넣을 수 있다. 하지만 버퍼가 가득 차 있을 경우는 어떻게 해야 할까? 이때 버퍼에 차 있는 데이터는 시간이 지나면 송신 인터럽트 처리에 의해 시리얼 포트로 전송된다.

즉, 시간이 지나면 인터럽트에 의해 자동으로 버퍼가 비게 된다. 그렇다면 버퍼가 가득 찬 상태에서는 송신 태스크를 대기상태로 만드는 것이 가장 좋은 방법일 것이다. 이 메커니즘도 세마포어를 사용해서 구현할 수 있다. 송신 인터럽트에서는 버퍼에서 읽어낸 데이터를 외부로 출력할 때마다 세마포어로 신호를 보내서 버퍼가 가득 차서 대기 상태에 있을지도 모를 태스크를 준비상태로 만든다. 대기 중인 태스크가 없을 경우는 단순히 세마포어 카운트 값만 증가해서 송신버퍼에 저장할 공간이 더 생긴 사실을 기록한다.

송신 태스크는 데이터를 버퍼에 저장하기 전에 세마포어 대기 함수를 먼저 호출한다. 버퍼에 빈 공간이 있을 경우(즉, 세마포어 카운트 값이 0이 아닐 경우)는 함수가 즉시 리턴해서 버퍼에 데이터를 저장한다.

반면에 버퍼가 가득 찬 경우(세마포어 카운트 값이 0일 경우)는 태스크가 대기 상태로 들어간다. 하지만 언젠가는 한 바이트 송신이 완료되어 송신완료 인터럽트가 발생한다. 송신 ISR에서는 버퍼에 저장된 데이터를 읽어내어 밖으로 출력하면서 세마포어로 신호를 보낸다. 이때 대기 중인 태스크가 깨어난다. 송신 메커니즘을 위해서는 수신 메커니즘과 반대로 송신 세마포어 초기값을 버퍼 크기로 초기화해야 한다. 다음은 송신과 관련

```

INT8U CommPutChar(INT8U c, INT16U to)
{
    INT8U oserr;

    OSSemPend(pbuf->RingBufTxSem, to, &oserr); /* 송신
버퍼에 빈 공간이 생기길 기다린다 */
    if(oserr == OS_TIMEOUT) {
        return(COMM_TX_TIMEOUT); /* 타임아웃 만료됨, 에
리코드 리턴 */
    }
    OS_ENTER_CRITICAL();
    pbuf->RingBufTxCtr++; /* 데이터 카운터를 증가시킨다
*/
    *pbuf->RingBufTxInPtr++ = c; /* 버퍼에 데이터를 저장
한다 */

    if(pbuf->RingBufTxInPtr == &pbuf-
>RingBufTx[COMM_TX_BUF_SIZE]) { /* 포인터 경계 처리 */
        pbuf->RingBufTxInPtr = &pbuf->RingBufTx[0];
    }
    if(pbuf->RingBufTxCtr == 1) { /* 첫 번째 데이터인
가? */
        ipend1 |= 0x01; /* 예, 송신 인터럽트 활성화 */
    }
    OS_EXIT_CRITICAL();
    return(COMM_NO_ERR);
}

INT8U CommGetTxChar(INT8U *err)
{
    INT8U c;

    if(pbuf->RingBufTxCtr > 0) { /* 버퍼에 데이터가 있는
가? */
        pbuf->RingBufTxCtr--; /* 예, 데이터 카운터를 감
소시킨다 */
        c = *pbuf->RingBufTxOutPtr++; /* 버퍼에서 데이
터를 읽는다 */
    }
}

```

리스트 8 <계속>

```

        if (pbuf->RingBufTxOutPtr == &pbuf->RingBufTx[COMM_TX_BUF_SIZE]) { /* 포인터 경계 처리 */
            pbuf->RingBufTxOutPtr = &pbuf->RingBufTx[0];
        }
        OSSemPost(pbuf->RingBufTxSem); /* 버퍼에 한 개
의 저장영역이 생겼음을 알린다 */
        *err = COMM_NO_ERR;
        return(c); /* 정상적으로 데이터를 읽었음 */
    } else {
        *err = COMM_TX_EMPTY;
        return(NUL); /* 버퍼가 비었음 */
    }
}

```

리스트 8

된 실제 코드를 보여준다(리스트 8).

시리얼 포트를 통해 데이터를 송신하고자 하는 태스크는 CommPutChar() 함수를 호출한다. 이 함수를 통해 한 바이트 단위로 데이터를 전송할 수 있다. 이 함수는 버퍼에 빈 공간이 있는지 점검한 뒤 공간이 있으면 데이터를 저장하는 일을 한다. 이를 위해 가장 먼저 하는 일은 OSSemPend()를 호출하는 것이다.

이때 세마포어의 내부 카운트 값이 0이 아닌 경우 즉, 버퍼가 하나 이상 비어있는 경우는 함수가 즉시 리턴해서 데이터를 버퍼에 저장하는 코드를 실행한다. 앞에서 설명한 것처럼 송신용 세마포어(RingBufTxSem)의 초기 카운트 값은 송신버퍼의 크기로 초기화되기 때문에 버퍼가 가득 차기 전에는 OSSemPend()를 호출해도 즉시 리턴한다. CommPutChar() 함수 호출로 인해 OSSemPend()가 호출될 때마다 세마포어의 내부 카운트 값은 하나씩 줄어든다. 따라서 OSSemPend()를 호출할 때 세마포어의 카운트 값이 0이어서 태스크가 대기상태로 들어가면 버퍼가 가득 찼다는 것을 의미한다.

버퍼가 가득 차 있는 상태에서 또 다른 데이터를 버퍼에 저장하려고 CommPutChar()를 호출할 경우 이런 상황이 발생한다. 예를 들자면, 송신 버퍼의 크기는 10바이트인데 15바이트의 패킷을 송신하기 위해 CommPutChar() 함수를 15회 연속으로

호출하는 경우다. 그럴 경우 10바이트까지는 태스크가 동작해서 버퍼에 저장하지만 11번째 데이터를 버퍼에 저장하려는 순간 태스크는 대기상태가 된다.

하지만 UART를 통한 최초 데이터 송신이 끝나면 송신 완료 인터럽트가 발생하고, 그 인터럽트 처리 루틴 안에서 버퍼에 있는 데이터를 하나 읽어오면서 세마포어로 신호를 보내므로 대기 중인 태스크는 잠시 깨어나서 11번째 데이터를 버퍼에 저장하고 다시 대기상태가 된다. 태스크가 15번째 데이터를 버퍼에 저장할 때까지 이런 과정을 반복하게 된다. 어쨌든 이런 경우가 발생하는 것은 바람직하지 않다. 메모리 제약이 심한 경우는 할 수 없겠지만, 그런 경우가 자주 발생하면 버퍼 크기를 늘리는 것이 좋다. OSSemPend()를 호출한 뒤 하는 일은 송신 데이터를 버퍼에 저장하는 것이다.

태스크가 버퍼에 접근하는 동안 다른 태스크 또는 인터럽트 서비스 루틴이 버퍼에 접근하는 것을 막기 위해 인터럽트를 금지하는 것을 주목하라. 물론 버퍼 접근을 마친 뒤에 인터럽트를 다시 해제하는 것은 기본이다. CommGetTxChar()는 송신 인터럽트 서비스 루틴에서 호출하는 함수다. 이 함수 앞뒤에 코드를 약간만 붙이면 그 자체로 인터럽트 서비스 루틴이 될 수도 있지만 인터럽트 서비스 루틴은 하드웨어와 RTOS에 의존적이므로 하드웨어와 RTOS에 무관한 부분을 별도의 함수로 만들어서 호출하는 방식을 사용하고 있다.

이 함수도 마찬가지로 세마포어 관련 함수만 바꾸면 다른 RTOS에서 얼마든지 사용할 수 있다. 이렇게 하드웨어에 무관한 형태로 함수를 별도로 만들어 두면 소프트웨어 이식성이 좋아진다. 함수 호출이 늘어나면서 약간의 오버헤드가 증가하지만 이런 소프트웨어는 다른 하드웨어 플랫폼에서도 쉽게 재사용할 수 있다는 큰 장점이 있다.

드라이버 사용 예

리스트 9는 시리얼 드라이버 사용법을 간단히 보여준다. CommTask() 태스크는 시리얼포트로부터 데이터를 주고받은 일을 담당한다. 이 태스크가 가장 먼저 해야 할 일은 시리얼 드라이버를 초기화하기 위해 CommInit()을 호출하는 것이다. CommInit() 호출 외에 필요한 다른 일을 한 후, 무한 루프로 진입해서 CommGetChar() 함수를 호출해서 시리얼 포트로부터 데이터를 읽는다.

```

void CommTask(void *pdat)
{
    INT8U err, sData;

    ...

    CommInit();
    ...

    for(;;) {
        sData = CommGetChar(0, &err);
        if(err != COMM_NO_ERR)
        {
            /* 수신 에러 처리 */
        }
        err = CommPutChar(sData, 0);
        if(err != COMM_NO_ERR)
        {
            /* 송신 에러 처리 */
        }
    }
}

```

리스트 9

이때 버퍼에 저장된 데이터가 없을 경우는 CommTask()가 CommGetChar()를 호출할 때 실행을 멈추고 대기상태로 들어간다. 대기상태로 들어간 이후 시리얼포트로 데이터가 수신돼서 인터럽트 서비스 루틴이 태스크를 깨워주면, CommGetChar()는 sData로 데이터를 전달하면서 리턴한다. 그 다음 CommPutChar()를 호출해서 수신 데이터를 그대로 송신한다. CommTask()는 이 동작을 무한 반복하면서 동작한다. 하지만 데이터가 수신되지 않으면 영원히 대기상태에 머물러 있기 때문에 CPU를 점유하지 않는다.

연재를 마치며

이번 호를 마지막으로 MicroC/OS-II 상에서의 디바이스 드라이버 작성법 연재를 마친다. 지금까지의 연재를 통해 RTOS

상에서 디바이스 드라이버를 구현할 때 알아야 할 핵심 개념을 설명했다고 생각한다. RTOS마다 고유한 I/O 서브시스템의 구조에 맞춰 디바이스 드라이버를 작성하고 등록하는 방법 등 다뤄야 할 주제가 더 있지만, 그런 주제는 RTOS에 매우 의존적이므로 특정 RTOS를 기준으로 설명하는 것은 이번 연재에서 제외했다. 그런 주제에 관심이 있는 독자는 참고서적[4]을 참조하기 바란다.

이번 연재에서 설명한 개념들은 실제 여러 RTOS용 디바이스 드라이버에 구현돼 있는 아주 중요한 것들이지만, 잘 설명된 문서나 책을 찾아보기 힘들다. 사실 초보 임베디드 엔지니어에게는 이런 개념 자체가 어려울 수도 있다. 기본적으로 RTOS와 세마포어에 대해 알고 있어야 하기 때문이다. 어쨌든 이번 연재가 RTOS를 처음 접하는 엔지니어, RTOS를 사용하고 있지만 개념을 정확히 파악하지 못하고 있는 엔지니어 모두에게 도움이 되기를 바란다.

사실 디바이스 드라이버를 비롯한 하드웨어 의존적인 코드를 전혀 건드리지 않고 업체에서 제공하는 BSP 상에서 작업하는 임베디드 엔지니어라면 이런 내용을 모르고 지내는 게 보통이다. 처음 연재에서도 말했듯이 필자는 그런 경우를 많이 보았다. RTOS가 제공하는 여러 가지 소프트웨어를 사용하기 위해서 RTOS를 사용하는 것 보다는 RTOS의 본질을 확실히 이해하고 RTOS의 기능을 충분히 활용해서 보다 효율적인 임베디드 시스템을 개발하기를 기대한다. 디바이스 드라이버를 작성해보는 것은 RTOS를 잘 이해할 수 있는 방법 중 하나가 될 수 있다.

마지막으로 연재 기사에서 더 궁금한 점이 있으면 whsung@dioiz.com으로 메일을 보내기 바란다. ^{Real}Time

저작권 및 참고자료

이 글의 일부 내용과 소스코드는 한국내 판권자인 (주)에이콘출판사의 허가를 받아 다음 서적에서 인용했습니다.

- [1] "MicroC/OS-II 실시간 커널, 제2판"(ISBN 898997533 -6)
- [2] "Embedded Systems Building Blocks, 2nd Edition"(ISBN 89-951540-4-7)
- [3] "C Programmer's Guide to Serial Communications", Joe Campbell
- [4] "RTOS를 이용한 실시간 임베디드 시스템 디자인", 번역서, 에이콘출판사